

Mixture of Experts

Many experts, only a few awake per token · video 2:13

This lesson answers one question: can a model have many more parameters without more compute per token? A **mixture of experts** (MoE) replaces each MLP with several expert MLPs and a **router** that sends every token to its top 1 or 2. In a tiny GPT, top-1 routing did not beat a dense model at the same compute, but top-2 routing beat a dense model of the same active size (1.596 vs 1.628), and a balancing loss kept all 8 experts busy.

How to use this guide: master one concept before moving to the next.

1. Watch the whole video once, without stopping.
2. Then take the concepts **in order**. For each one: read the short explanation, look at the picture from the video (▶ is the time in the video), and answer every question in its *Check* without looking anything up.
3. Check your answers in the **Answers** section at the end (page 5).
4. **Pass mark: every question in the check correct.** If you missed one, rewatch that concept's video segment, reread the explanation, and redo only the questions you missed. Tick the concept off below, then move on.

It is strongly suggested that you **master each concept before starting the next one**. Each concept builds on the previous ones.

Concepts in this lesson

CONCEPT	VIDEO SEGMENT	QUESTIONS	MASTERED
1. Experts and a router	▶ 0:08-0:51	2	☐
2. Top-1, balancing, top-2	▶ 0:51-1:41	3	☐
3. The deal, and the code	▶ 1:41-2:04	1	☐

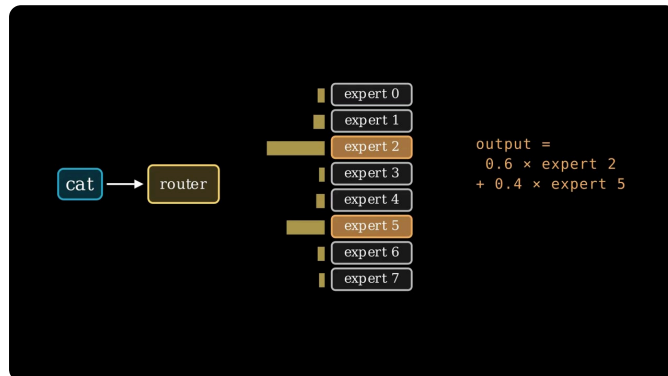
Before you start: Deep Dive episode 15 (the MLP and its activation) and episode 23 (scaling laws). Code: `code/d28_moe` (PyTorch only; five tiny models, about an hour on a CPU).

CONCEPT 1 OF 3

Experts and a router

▶ Video 0:08-0:51

A **mixture of experts** has many more parameters than it uses for each token. Mixtral 8x7B, for example, has 8 experts per layer and uses 2 for each token. The MLP in each block is replaced by several expert MLPs plus a small **router** (one linear layer and a softmax). For every token, the router scores the experts, keeps the best k, and mixes their outputs by those (renormalized) scores; the other experts don't run at all.



our tiny GPT, 8 experts in every layer

dense:	818,241 parameters,	818,241 used per token
MoE, top-1:	4,510,273 parameters,	822,337 used per token

5.5x the parameters, the same compute per token

▶ 0:37 The router scores the experts, keeps the best one or two, and mixes their outputs; the others don't run (scores illustrative).

▶ 0:50 8 experts per layer: 5.5x the parameters, the same compute per token with top-1.

In our tiny GPT, 8 experts per layer make **4,510,273** parameters, 5.5 times the dense model's 818,241. With top-1 routing each token still uses about the same as before: **822,337** (the router adds a little).

KEY IDEA

Parameters and compute are decoupled: total size grows with the number of experts, compute per token with k.

Check 1: Experts and a router

Answer all 2 questions before looking at the answers. Pass mark: all correct.

1.1 CALCULATION

Router scores for two chosen experts are 0.41 and 0.27. With what weights are their outputs mixed?

1.2 CALCULATION

One expert MLP here has 131,712 parameters. With top-2 instead of top-1, how many more parameters are active per token across the 4 layers?

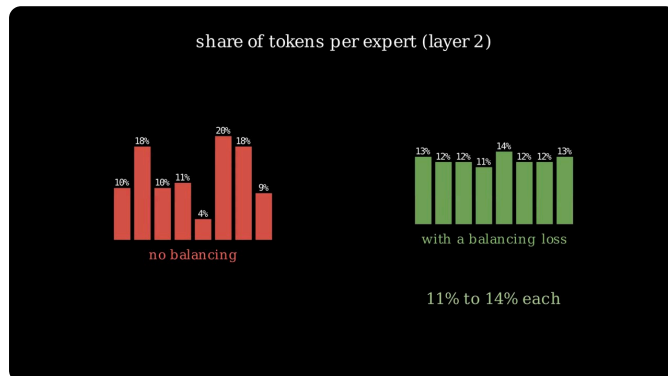
CONCEPT 2 OF 3

Top-1, balancing, top-2

▶ Video 0:51-1:41

After 2,000 steps, the dense model reaches **1.644** and MoE with top-1 routing **1.687**: worse. At this small scale and short training, each expert sees only an eighth of the tokens, and it doesn't pay off.

Without help, the router also plays favorites: in layer 2 one expert gets 20% of the tokens, another only 4%. A small **load-balancing loss** (weight 0.01) that rewards even use fixes it: every expert then gets 11% to 14% (loss 1.695).



▶ 1:22 Without balancing the router plays favorites (4% to 20% of tokens); with a balancing loss, 11% to 14%.



▶ 1:40 Top-2 MoE 1.596 vs a dense model of the same active size 1.628; top-1 1.687 vs dense 1.644.

With **top-2** routing the loss drops to **1.596**, the best of all. But each token now uses 1.35 million parameters, so the fair comparison is a dense model just as big: an MLP twice as wide (1.34 million) reaches **1.628**. The mixture still wins.

KEY IDEA

Compare MoE with a dense model of the same compute per token; here top-2 won that comparison and top-1 did not.

Check 2: Top-1, balancing, top-2

Answer all 3 questions before looking at the answers. Pass mark: all correct.

2.1 PUT IN ORDER

Order from best to worst: *dense · MoE top-1 · MoE top-2 · dense twice as wide*.

2.2 TRUE OR FALSE

“Without a balancing loss, the router sends tokens evenly to all experts.”

True ☐ False ☐

2.3 SHORT ANSWER

Why is a dense model “twice as wide” the fair baseline for top-2 MoE?

CONCEPT 3 OF 3

The deal, and the code

▶ Video 1:41-2:04

That is the deal: at the same compute per token, a mixture of experts can hold far more parameters, and it can come out ahead. The price is memory for all the experts (every one must be loaded, even if few run) and some care to keep them all busy. In code, the router is a linear layer and a softmax, `topk` picks the experts, and each expert runs only on the tokens sent to it.

same compute per token, far more parameters
✗ memory for all the experts

▶ 1:53 Same compute per token, far more parameters; the price: memory for all experts and care to keep them busy.

```
1 probs = router(x).softmax(-1)           # (tokens, 8 experts)
2 top_p, top_i = probs.topk(2, dim=-1)    # the 2 best experts per token
3 for e in range(8):
4     rows = (top_i == e).any(-1)         # tokens sent to expert e
5     out[rows] += w[rows] * experts[e](x[rows])
```

▶ 2:02 Router, top-k, and each expert run only on the tokens sent to it.

KEY IDEA

MoE trades memory for compute: big in storage, small in work per token.

Check 3: The deal, and the code

Answer all 1 questions before looking at the answers. Pass mark: all correct.

3.1 MULTIPLE CHOICE

Which cost does a mixture of experts NOT reduce?

- A. Compute per token
- B. Memory to hold the model
- C. Active parameters per token
- D. None of these

ANSWERS

Answers and explanations

Mark each answer right or wrong. If a concept has any wrong answer, rewatch its segment (▶) and redo the questions you missed before moving on.

1. Experts and a router · rewatch ▶ 0:08-0:51

1.1 About 0.60 and 0.40. Renormalized: $0.41 / 0.68$ and $0.27 / 0.68$.

1.2 526,848. $4 \text{ layers} \times 1 \text{ extra expert} \times 131,712 = 526,848$ ($1,349,185 - 822,337$).

2. Top-1, balancing, top-2 · rewatch ▶ 0:51-1:41

2.1 MoE top-2 (1.596) → dense twice as wide (1.628) → dense (1.644) → MoE top-1 (1.687). From the episode's runs.

2.2 False. Shares ranged from 4% to 20% in layer 2.

2.3 It uses about the same number of active parameters (compute) per token, 1.34 million vs 1.35 million, so any difference comes from MoE's extra total parameters, not extra compute. Comparing top-2 MoE with the small dense model would mix two effects.

3. The deal, and the code · rewatch ▶ 1:41-2:04

3.1 B. All experts must be stored, even though only k run per token.